LABNEXUS

A report submitted in partial fulfillment of the Academic requirements for the award of the degree of

Bachelor of Technology

A LAN-BASED COMPUTER LAB MANAGEMENT SYSTEM

Abhigyan Tiwari (25H51A67D0)

K. Bhargavi (25H51A67F7)

N. Chandrika (25H51A67H2)

B. Ashwitha (25H51A6716)

K. Sri Vardhan (25H51A6717)

UNDER THE COURSE

SOCIAL INNOVATION AND ENTREPRENEURSHIP



CENTRE FOR ENGINEERING EDUCATION RESEARCH

CMR COLLEGE OF ENGINEERING & TECHNOLOGY
(Autonomous)

(NAAC Accredited with 'A^{+}' Grade & NBA Accredited)
(Approved by AICTE, Permanently Affiliated to JNTU Hyderabad)
KANDLAKOYA, MEDCHAL ROAD, HYDERABAD-501401

2026-27

CENTRE FOR ENGINEERING EDUCATION RESEARCH

CMR COLLEGE OF ENGINEERING & TECHNOLOGY
(Autonomous)
(NAAC Accredited with 'A^{+}' Grade & NBA Accredited)
(Approved by AICTE, Permanently Affiliated to JNTU Hyderabad)
KANDLAKOYA, MEDCHAL ROAD, HYDERABAD-501401



CERTIFICATE

This is to certify that the report entitled "LabNexus — A LAN-Based Computer Lab Management System" is a bonafide work done by Abhigyan Tiwari (25H51A67D0), K. Bhargavi (25H51A67F7), N. Chandrika (25H51A67H2), B. Ashwitha (25H51A6716) and K. Sri Vardhan (25H51A6717) of II B.Tech, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, submitted to the Centre for Engineering Education Research, CMR College of Engineering & Technology, Hyderabad, during the academic year 2026-27.

Project Coordinator
[COORDINATOR-1 NAME]
[COORDINATOR-2 NAME]
[COORDINATOR-3 NAME]

Head of the Department (CEER)
Mr. B. Suresh Ram

DECLARATION

We, the students of II B.Tech of the Centre for Engineering Education Research, CMR College of Engineering & Technology, Kandlakoya, Hyderabad, hereby declare that under the supervision of our course coordinators we have independently carried out the project titled "LabNexus — A LAN-Based Computer Lab Management System" and submitted this report in partial fulfilment of the requirement for the award of Bachelor of Technology by the Jawaharlal Nehru Technological University, Hyderabad (JNTUH) during the academic year 2026-27.

Place: Hyderabad Date: _____

STUDENT NAME	ROLL NUMBER	SIGNATURE
Abhigyan Tiwari	25H51A67D0	
K. Bhargavi	25H51A67F7	
N. Chandrika	25H51A67H2	
B. Ashwitha	25H51A6716	
K. Sri Vardhan	25H51A6717	

Internal Examiner's signature **External Examiner's signature**

ACKNOWLEDGEMENT

We are obliged and grateful to thank Mr. B. Suresh Ram, Head (CEER), CMRCET, and our project coordinators for their cooperation in all respects during the course of this project.

We would like to thank Dr. K. Srinivasa Rao, Principal, CMRCET, for his constant encouragement, valuable guidance and support.

We thank all our faculty members and laboratory assistants for their timely help and valid support during the development and testing of this project.

We own all our success to our beloved parents, whose vision, love and inspiration has made us what we are today.

ABSTRACT

LabNexus is a low-cost local-area-network system for managing college computer labs. A Node.js / Express server on the lab staff's laptop serves a browser dashboard (default port 8000); a lightweight Node.js agent on every lab PC discovers the server through UDP broadcast, enrolls once with a server-issued seat ID and token, and sends periodic authenticated heartbeats. From the dashboard the administrator sees honest seat state — fresh evidence only, UNKNOWN when evidence is stale — and can lock, unlock, grant time-limited access, message, sleep, reboot or Wake-on-LAN each seat. A validated weekly timetable is followed automatically: periods open and close at their boundaries and attendance-based allocation opens exactly the required number of online seats, with grants that expire at period end. Optional AI-assisted extraction analyses an uploaded timetable photograph through an OpenAI-compatible vision endpoint (a free Gemini key works) and only proposes periods for review. All state and images stay local by default and every login, command, grant and period transition is written to an audit log. The system was verified end-to-end with a 206-check integration suite.

Access is enforced on the machine, not just on the server: every lab PC runs a full-screen overlay and validates each server command against its seat token, while the fail-closed agent re-locks a machine that loses contact with the server. All logins, grants, commands and period transitions are recorded in a local audit log, so every change can be traced afterwards.

TABLE OF CONTENTS

CHAPTERS		DESCRIPTION	PAGE No
		Abstract	v
1		Introduction	1-2
2		Literature Review	3-7
3		Problem Definition	8-10
	3.1	Problem Statement	8
	3.2	Objectives	8
	3.3	Requirement Analysis	8
4		Methodology	11-14
	4.1	Conceptual Design	11
	4.2	Block Diagram	12
	4.3	Design Description	12
5		Implementation	15-19
	5.1	Results and Discussion	15
	5.2	Conclusion	18
6	6.1	Appendix	20
	6.2	References	21
	6.3	Source Code	22
7		Team Details	23

CHAPTER-1
INTRODUCTION

1.1 Overview

LabNexus is a local-area-network (LAN) lab-computer management system that lets a single lab in-charge watch and control every PC in a computer laboratory from one browser dashboard, with no recurring licence cost and no dependency on the public internet.

An administrator server runs on the lab staff's laptop; a lightweight Node.js agent on each lab PC discovers it through a UDP broadcast, registers with a server-issued seat ID and token, and then reports authenticated heartbeats. From the dashboard the administrator can lock or unlock a machine, grant time-limited access, message a student, put a seat to sleep, reboot it or wake it with Wake-on-LAN, and every action is recorded in a local audit log.

Access follows the confirmed weekly timetable: periods open and close automatically, attendance allocates exactly as many seats as the number of students present, grants expire and re-lock by themselves, and a fail-closed agent keeps a disconnected machine locked. A 206-check integration suite verifies these workflows end to end.

1.2 Scope of the Project

The system is intended for a single computer laboratory of a college department. Its scope covers:

- discovery, enrolment and monitoring of the lab PCs without manual IP configuration;
- lock, unlock, grant, message, sleep, reboot and Wake-on-LAN control of every seat;
- a validated weekly timetable with automatic period boundaries;
- attendance-based allocation of exactly the required number of seats, with automatic expiry;
- password-protected administration and a complete, searchable audit trail;
- optional AI-assisted extraction of timetable photographs for administrator review.

The system does not replace the college ERP, does not maintain student marks and does not require internet access for its core functions; only the optional timetable extraction uses an external vision API.

1.3 Significance of the Project

A college laboratory is shared by several batches, and the person responsible for it is usually also teaching. Manual preparation therefore consumes time that should be spent on instruction, and the absence of records makes it difficult to answer even simple questions about when a room was opened or which students used a seat.

LabNexus addresses both problems at once. The laboratory follows its own published timetable, seats are opened only for the students who are actually present, and every change is written to a log that the department can inspect. Because the system is self-hosted and runs on the existing LAN, it remains affordable for a department that cannot fund a per-seat licence.

- The laboratory opens and closes on time without a staff member walking the room.
- Seats are granted for the exact number of students present, which discourages unauthorised use of idle machines.
- The audit log provides evidence for internal review and for resolving disputes.
- No recurring cost is incurred and no student data leaves the campus network.

1.4 Organisation of the Report

Chapter 2 reviews existing approaches to laboratory management and identifies their gaps. Chapter 3 states the problem and derives the functional, hardware and software requirements. Chapter 4 explains the methodology and the design of the system. Chapter 5 presents the implementation results, the verification performed and the conclusion. Chapter 6 contains the appendix, the references and source-code excerpts, and Chapter 7 lists the team details.

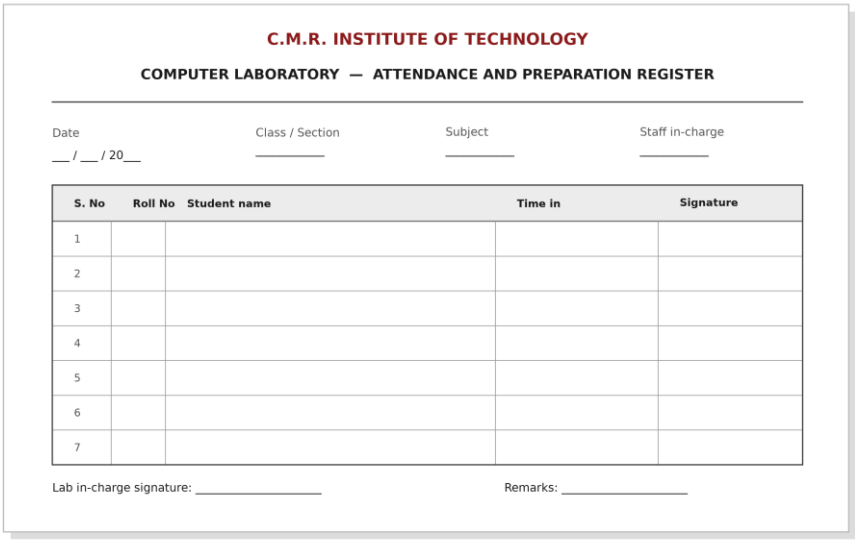
CHAPTER-2
LITERATURE REVIEW

2.1 Existing Solutions

Computer laboratories in colleges are managed today in one of three ways: by hand, with the operating system's own policy tools, or with a commercial laboratory-management suite. Each approach is reviewed below, together with the features it offers and the limitations that matter in a college timetable environment.

2.1.1 Manual Lab Management

Most computer laboratories are managed manually: the lab in-charge walks the room, checks each machine, signs a preparation or attendance sheet and locks the machines one by one at the end of a session. The paper register that records this work is shown in Figure 2.1.1.



C.M.R. INSTITUTE OF TECHNOLOGY

COMPUTER LABORATORY — ATTENDANCE AND PREPARATION REGISTER

Date __ / __ / 20__ Class / Section _____ Subject _____ Staff in-charge _____

S. No	Roll No	Student name	Time in	Signature
1				
2				
3				
4				
5				
6				
7				

Lab in-charge signature: _____ Remarks: _____

FIG. 2.1.1: Manual laboratory attendance and preparation register (representative form)

The approach needs no new software, but it scales poorly. Turning many machines on, locking them between periods and preparing the room for the next class all depend on a person being present and remembering every step.

There is no central record of what happened — who opened the room, which students attended, when a machine was unlocked — so later disputes are difficult to resolve and no report can be produced from the register without transcribing it by hand.

Features

- Zero cost: no new software or hardware is required.
- Simple: any staff member can follow the procedure without training.
- Direct supervision: a person sees the actual condition of each machine.

Limitations

- Slow: preparing a whole laboratory by hand takes many minutes.
- Inconsistent: different staff members lock and unlock differently.
- No audit trail: paper records are easy to lose and hard to search.
- No timetable enforcement: rooms remain as the last user left them.
- No remote view: nothing is visible away from the laboratory.
- Attendance is disconnected from the state of the machines.

2.1.2 Operating-System and Group-Policy Tools

Windows and Linux offer per-machine controls: screen-lock policies, scheduled shutdown scripts, local accounts and, on managed domains, group policies that enforce settings across many machines at once.

These tools can switch machines off at a fixed time or lock screens inside a domain, but they know nothing about a specific lab period, a specific class or the seats that a particular batch is entitled to use.

Temporary changes — an extra evening batch or an extended lab period — still require manual intervention, and nothing reports which seats are open at any moment. Figure 2.1.2 shows how such a policy reaches every machine and why it cannot follow a timetable.

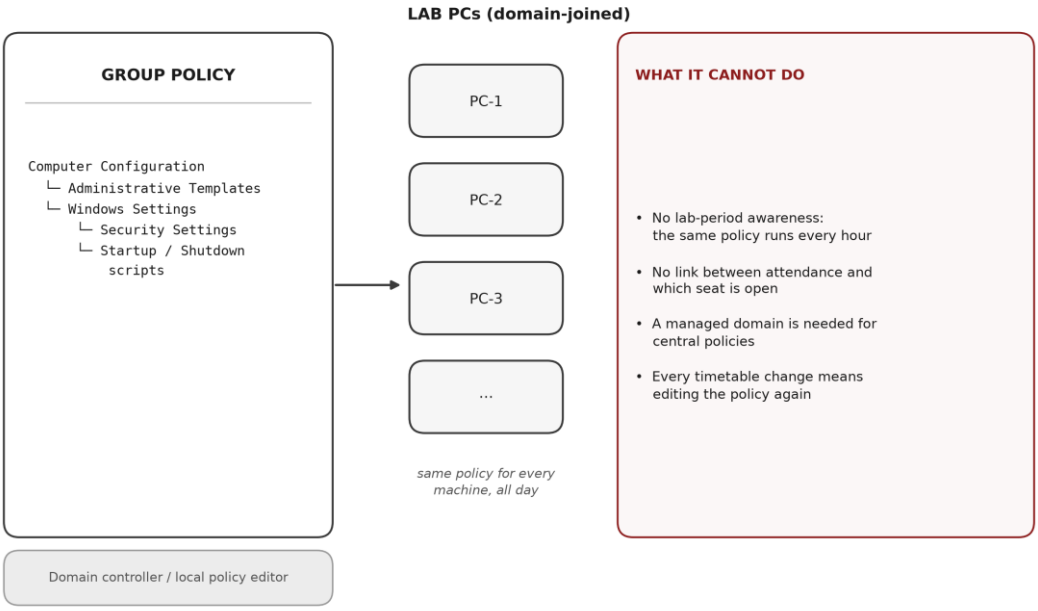


FIG. 2.1.2: Operating-system group policy applied to every lab PC (schematic)

Features

- Standard tooling: built into the operating system.
- Policy enforcement: screen locks can be enforced centrally.
- Scheduling: fixed-time shutdowns and startups are possible.
- Well documented and widely supported.

Limitations

- Coarse grain: applies to machines, not to specific lab periods.
- A managed domain is required for group policies.
- No attendance link: presence is not connected to access.
- A static schedule must be edited whenever the timetable changes.
- No seat-level view of which machine is open now.

2.1.3 Computer-Lab Management Software Suites

Laboratory-management suites connect every machine to a teacher console and offer screen broadcast, remote control, application blocking and session timers across the room. They solve the room-level control problem well and are widely used in computer training centres.

Licences are priced per seat, however, and their activity focus rarely matches a college timetable with changing batches and attendance-based seat allocation.

They also assume reliable, always-on networking and dedicated administration, which small laboratories may not have. A typical teacher console is illustrated in Figure 2.1.3.

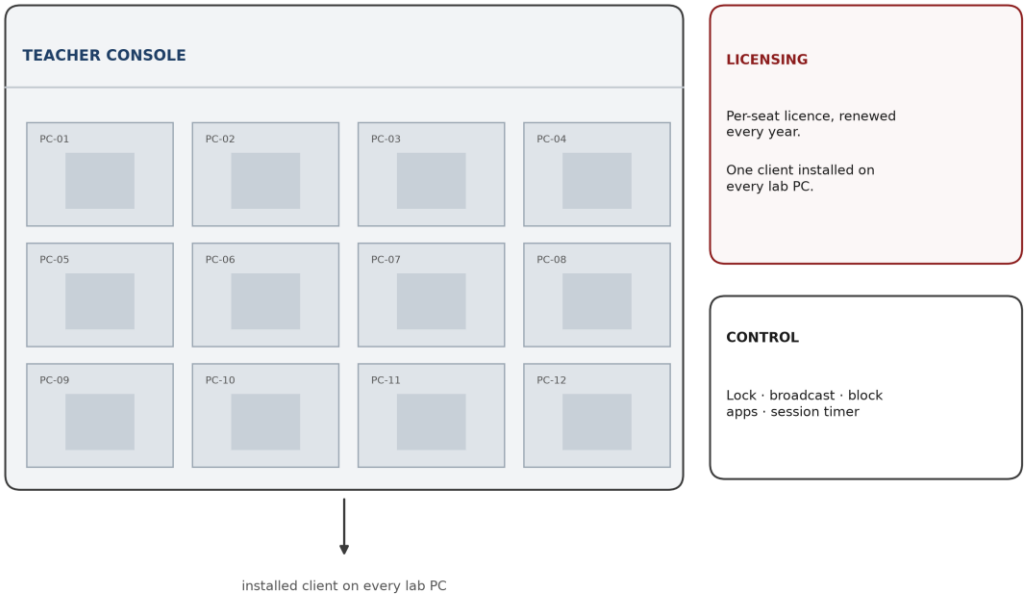


FIG. 2.1.3: Typical commercial laboratory-management suite console (schematic)

Features

- Room-wide control: lock, unlock and message all seats at once.
- Screen broadcast: push a demonstration to every monitor.
- Timed sessions: examination-style sessions end automatically.
- Attendance counters, application blocking and usage statistics in some products.

Limitations

- Licence cost: priced per seat and usually renewed every year.
- Heavy client software installed on every machine.
- No weekly college timetable model.
- Control stops when the LAN or the licensing server is unavailable.
- Screen-watching features may not suit every laboratory.

2.2 Gaps in Existing Solutions

All the approaches above leave the same gaps for a college laboratory, and those gaps motivated the present work:

- No low-cost option that works without a managed domain or paid licences.
- No attendance-linked allocation: seats are not opened for exactly N students.
- No weekly timetable awareness with automatic period opening and closing.
- No honest evidence: tools report what was ordered, not what a seat actually shows.
- No local audit log of who did what to which machine and when.
- Existing suites assume stable, always-managed networks and constant support.
- Paper-based attendance remains disconnected from the state of the machines.

2.3 Proposed Solution

LabNexus therefore proposes a self-hosted, zero-licence LAN system that combines room-level control with the timetable and attendance logic that a college laboratory actually uses:

- A central dashboard that shows every seat with fresh evidence.
- UDP auto-discovery so new machines join without IP configuration.
- Time-limited grants that expire by themselves and re-lock the seat.
- A weekly timetable follower that opens and closes periods on time.
- Attendance-based allocation that opens exactly the number of seats required.
- An in-built audit log of logins, grants, commands and period changes.

CHAPTER-3
PROBLEM DEFINITION

3.1 Problem Statement

Operating a college computer laboratory means opening many machines for one class, locking them between periods, waking them for the next batch and accounting for every change — all done by hand today, with no record of what happened and no budget for a commercial suite. The problem is to perform all of these operations from one dashboard on the campus LAN using only ordinary PCs and a staff laptop, and to remain safe when the network or an agent misbehaves.

3.2 Objectives

The objectives of the project are:

- To provide a single, evidence-based view of every seat in the laboratory.
- To open and close seats automatically according to the confirmed timetable.
- To allocate seats from attendance and to release them at the end of a period.
- To ensure security and reliability through token authentication and fail-closed behaviour.
- To reduce repetitive manual work for the laboratory staff and to keep a complete audit trail.

3.3 Requirement Analysis

The requirements were derived from the problem statement and from the day-to-day routine of a college laboratory. They are grouped into functional, hardware and software requirements.

3.3.1 Functional Requirements

1. The system shall discover and register lab PCs without manual IP configuration.
2. The system shall display the observed state of every seat from fresh, authenticated evidence.
3. The system shall lock, unlock, grant, message, sleep, reboot and wake a seat.
4. The system shall enforce access on the machine itself, independently of the server connection.

5. The system shall store and validate a weekly timetable and open and close periods automatically.
6. The system shall allocate exactly the required number of seats from attendance and expire them at period end.
7. The system shall authenticate administrators and agents and shall audit every action.
8. The system shall operate on the campus LAN without internet access.

3.3.2 Hardware Requirements

The hardware required by the system is summarised in Table 3.1. No special-purpose hardware is needed: the laboratory uses the computers and the network that already exist.

Table 3.1: Hardware requirements

Component	Minimum specification	Purpose
Administrator computer	Laptop or desktop, 4 GB RAM, Node.js 18 or later	Runs the Express server, dashboard, discovery responder and audit log
Lab computers	Any PC that can run Node.js 18 or later and Windows 10/11	Runs the agent and the full-screen lock overlay
Network	Switched LAN, 100 Mbps or faster; wired seats preferred	Carries HTTP polling, UDP discovery and Wake-on-LAN packets
Network interface cards	Wake-on-LAN support enabled in BIOS/UEFI and driver	Allows a powered-off seat to be woken by a magic packet
Server storage	2 GB free disk space on the administrator computer	Stores the JSON state file, timetable photographs and audit log
Power supply	Standard laboratory power with UPS backup (recommended)	Prevents an unexpected shutdown from interrupting a period

3.3.3 Software Requirements

The software requirements are listed in Table 3.2. All components except the optional vision API are free and open source, and none of them requires a licence.

Table 3.2: Software requirements

Software	Version / availability	Purpose
Windows 10 or 11	Installed on the lab PCs	Hosts the agent and the lock overlay
Node.js	18 LTS or later	Runtime for the server and the agent
Express	4.x (installed from npm)	HTTP API and static dashboard hosting
Web browser	Current Chrome, Edge or Firefox	Displays the administrator dashboard
Google Gemini API (optional)	Free API key from Google AI Studio	Reads timetable photographs; the core system works without it

3.3.4 Design Constraints

The requirements are bounded by the conditions in which a college laboratory actually operates. These constraints shaped several design decisions described in the next chapter:

- The core system must work without internet access; only timetable extraction may use an external vision API.
- It must run on the existing Windows PCs and the staff laptop, with no additional server hardware and no licence fee.
- A lab PC may be switched off or disconnected at any moment, so access must fail closed rather than fail open.
- Student data must remain on the campus network and be retained only in the local audit log.
- The dashboard must be usable by a single lab in-charge without dedicated training.

CHAPTER-4
METHODOLOGY

4.1 Conceptual Design

LabNexus solves the problem of coordinating many lab PCs from one place without buying a commercial suite or maintaining a domain, using a hub-and-agent architecture over the existing campus LAN. The arrangement is shown in Figure 4.1.1.

The conceptual design is deliberately simple: one administrator server owns the truth, and every lab PC runs a small outbound-polling agent. The server never initiates a connection into a lab machine — which avoids firewall and NAT problems — and an agent never trusts a command that does not arrive with the right token.

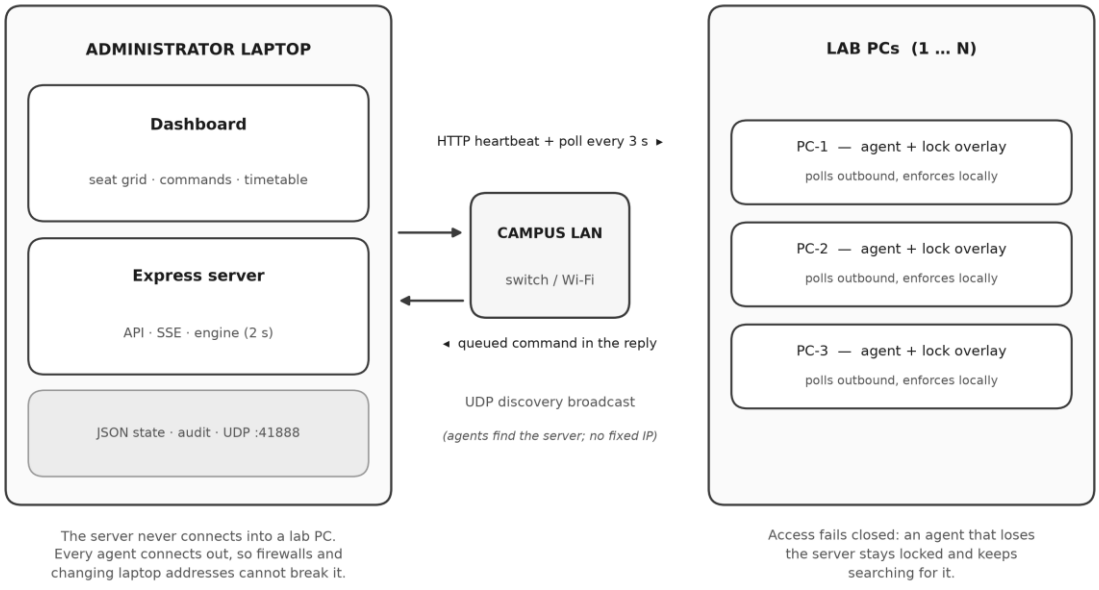


FIG. 4.1.1: LabNexus conceptual design (server and agents)

The mechanism of the solution is equally simple. First the server is located: the agent broadcasts “LABNEXUS_DISCOVER_V1” on the LAN and the server answers with its address. The agent then enrols once and receives a unique seat ID and a random, server-issued token.

From then on the agent loop consists of heartbeats and polls: it sends an authenticated heartbeat every few seconds, pulls any queued command, executes it (lock overlay, unlock, sleep, reboot, message or wake) and reports enforcement back, while the engine expires grants exactly on time and audits every state change.

4.2 Block Diagram

Figure 4.2.1 shows the block diagram of the system. The dashboard talks to the Express server over HTTP and receives live updates through Server-Sent Events. The server keeps the authoritative state in a JSON file, runs the two-second reconcile loop and answers UDP discovery requests. Each lab PC runs an agent that polls the server, enforces the seat state locally and reports what it actually enforces.

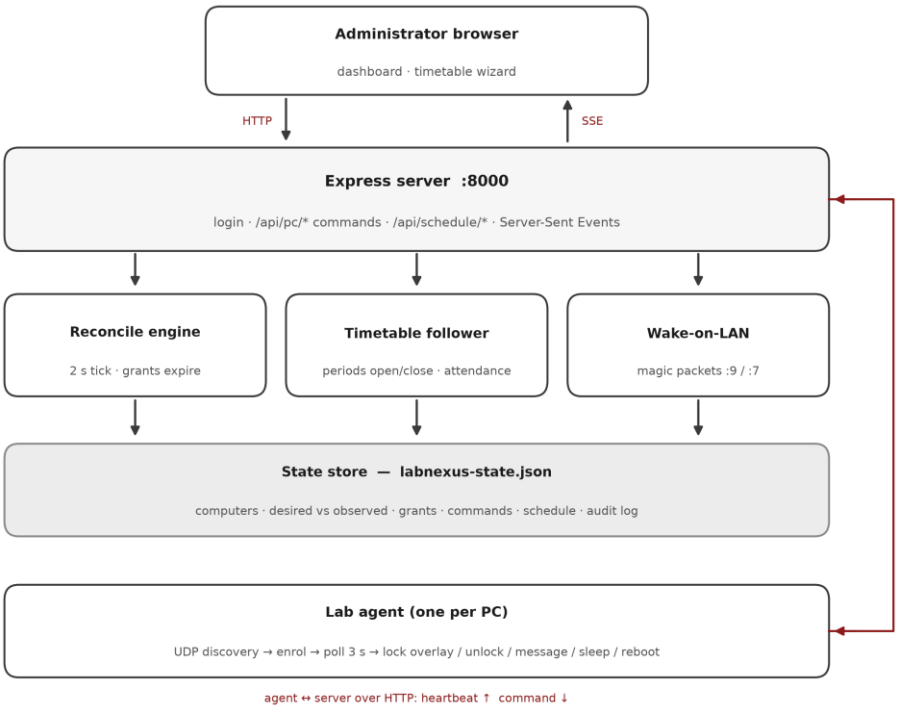


FIG. 4.2.1: System block diagram (dashboard — server — LAN — agents)

4.3 Design Description

The design closes the loop around the server tick: the dashboard shows every seat with its observed state and a one-click command bar; each two-second tick recomputes evidence and dispatches queued commands; grants carry an expiry and remain enforceable locally even if the

server restarts; Server-Sent Events push live updates; and Wake-on-LAN and the lock overlay plug into the same command path. Figure 4.3.1 shows how the server and agent modules fit together.

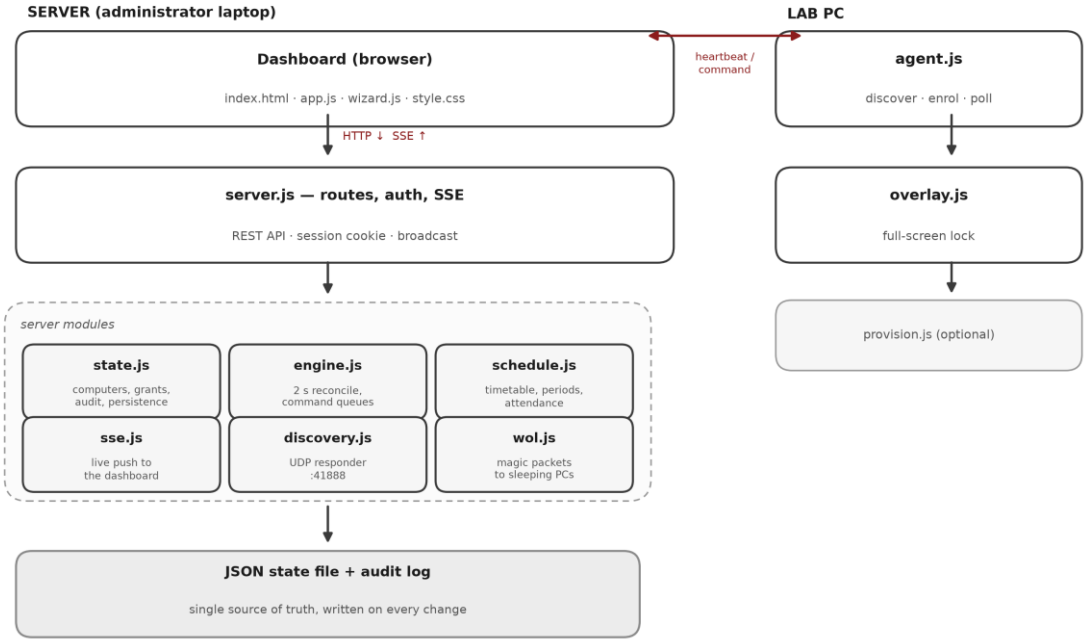


FIG. 4.3.1: Design and integration of the LabNexus modules

4.3.1 Evidence-Based State Model

Central to the design is the evidence model: the observed state of a seat is computed only from fresh, authenticated heartbeats, and anything else is displayed as UNKNOWN rather than guessed. The desired state (what the administrator wants) is stored separately from the observed state (what the agent reports), so a queued command is never mistaken for a completed transition.

4.3.2 Timetable and Attendance

The weekly timetable module validates the entered periods and follows them every day, opening and closing periods at their boundaries in the server tick. It handles back-to-back periods correctly, so a seat granted in period A is re-locked at the boundary instead of rolling into period B. Attendance allocation opens exactly as many seats as there are students present, reports a

clear shortage when there are not enough fresh seats, and expires every grant at the end of the period.

4.3.3 Security and Auditability

Administrators sign in with a password and receive an HttpOnly, SameSite-strict session cookie that expires after twelve hours. Agents authenticate every request with their per-seat token, and a second machine that claims an existing seat is refused with HTTP 409. Logins (including failed attempts), grants, commands, schedule changes, allocation results and period transitions are all written to the local audit log with timestamps.

4.3.4 Fail-Closed Operation

If an agent cannot reach the server, the machine re-locks locally and keeps searching for the server. If evidence goes stale, the seat is shown as UNKNOWN and a grant cannot silently persist. Data stays local by default inside a JSON state file; only the optional, per-request timetable extraction ever leaves the machine.

CHAPTER-5
IMPLEMENTATION

5.1 Results and Discussion

The system was implemented in Node.js and verified with an automated integration suite that boots a throwaway server and several simulated agents. The suite exercises the happy paths and the negative paths described below; all 206 checks pass. The dashboard that an administrator uses during a period is shown in Figure 5.1.1.

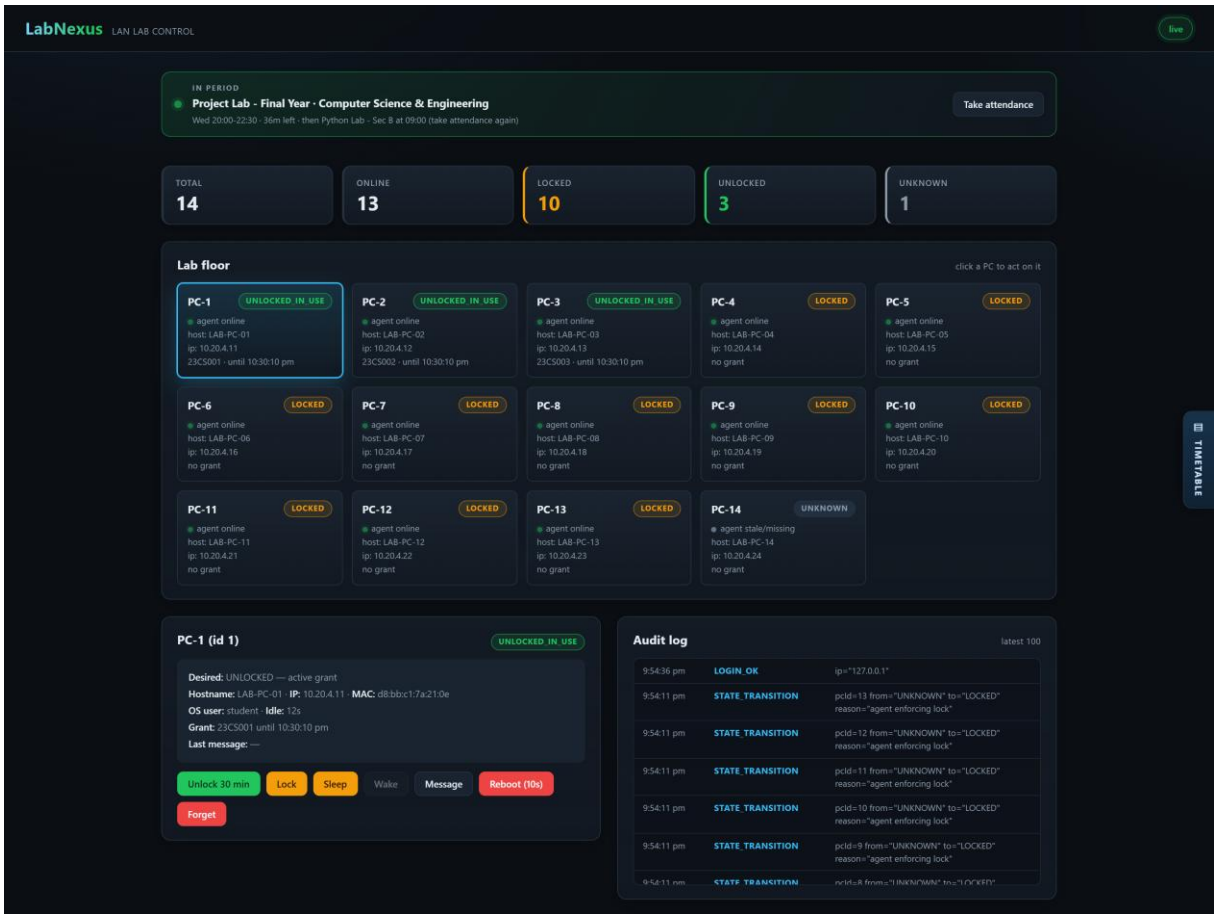


FIG. 5.1.1: Administrator dashboard: live seat grid, selected seat and audit log

5.1.1 Seat Locking and Command Delivery

Seat commands (lock, unlock, message, sleep and reboot) reached fresh agents within one poll cycle. A lock covered enqueue, poll, overlay display and enforcement report in a few seconds, and every step left an audit entry.

Wake-on-LAN packets were emitted for powered-off seats, but a seat was reported awake only when a fresh heartbeat actually returned. A network card without Wake-on-LAN enabled was therefore shown honestly as still down instead of optimistically awake.

5.1.2 Timetable and Attendance

The administrator confirms the weekly lab hours once and the laboratory then follows them without further intervention; the confirmed grid is shown in Figure 5.1.2.

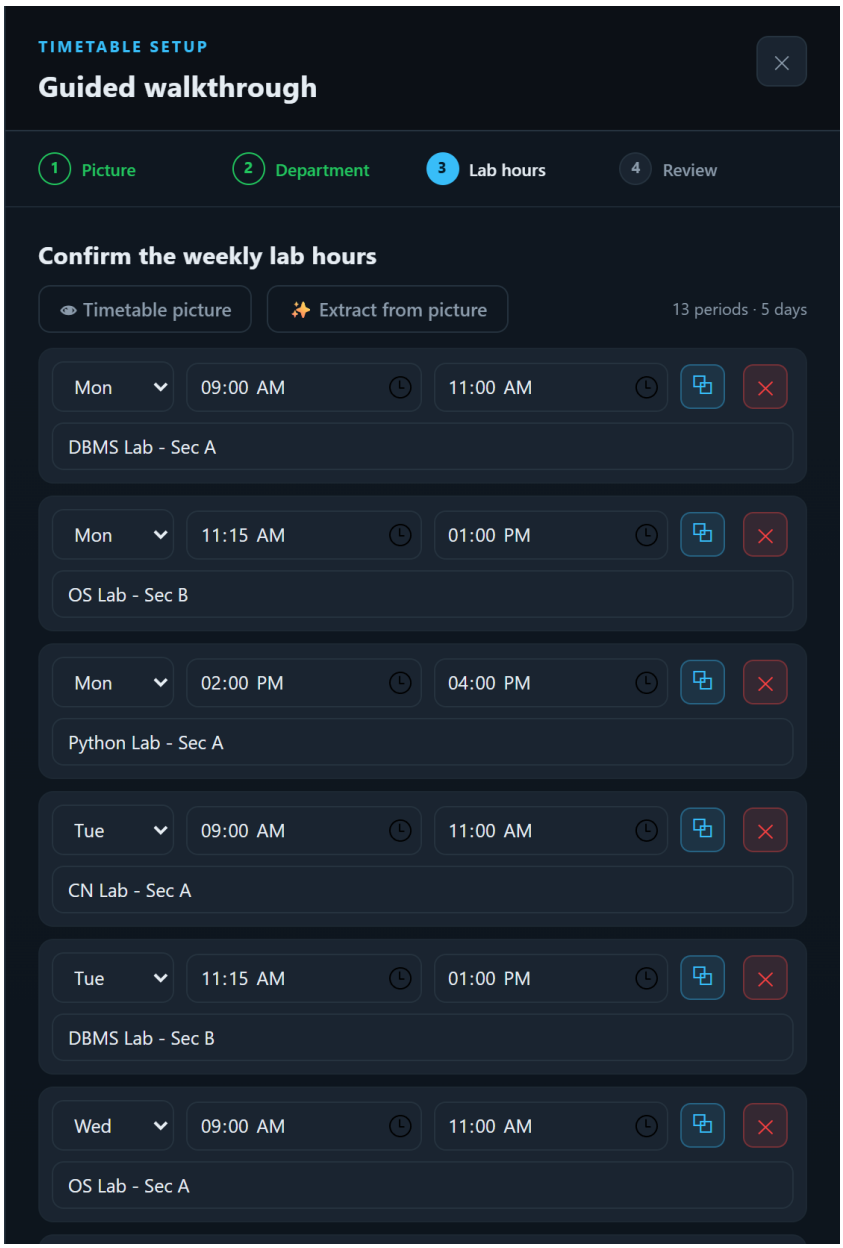


FIG. 5.1.2: Timetable setup: the confirmed weekly lab hours

At period boundaries the scheduler closed the ending period and opened the next one in the same server tick; a grant from period A never rolled into period B. After the last period the room re-locked itself with no leftover grants.

Attendance through the schedule endpoint allocated one seat per student present, using only fresh agents, reported a clear shortage when seats were insufficient, and expired every allocation at period end. The audit log recorded each allocation and each refusal. Figure 5.1.3 shows three roll numbers opening exactly three seats for the period.

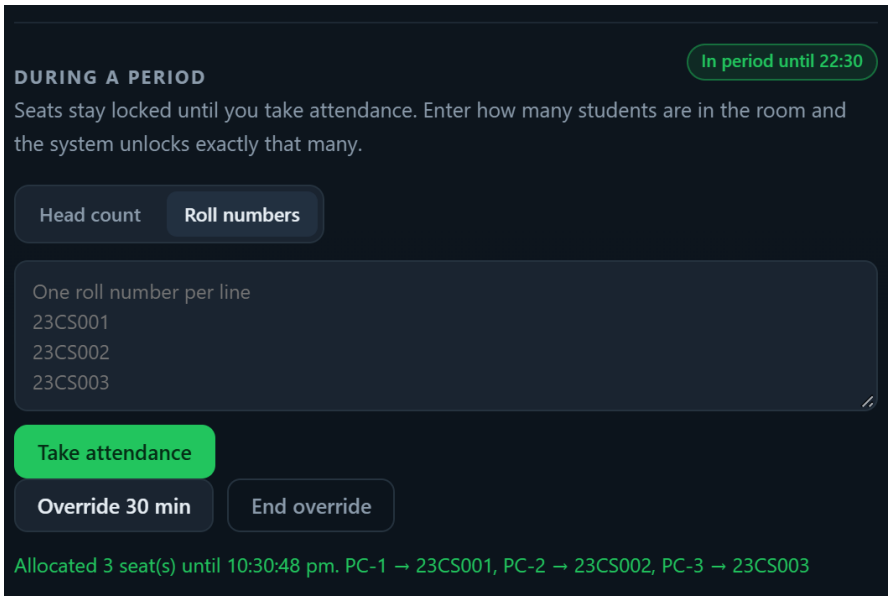


FIG. 5.1.3: Attendance allocation: three roll numbers, three seats opened for the period

5.1.3 Security and Failure Recovery

Security was exercised negatively as well: agent endpoints rejected requests with a missing or wrong token (HTTP 401) and refused duplicate seat registrations (HTTP 409), while administrator routes without a session cookie answered HTTP 401 Unauthorized.

Failure recovery worked without intervention. A gap in the network left seats shown as UNKNOWN rather than as stale truth, and agents that went silent re-enrolled cleanly and had their state corrected from fresh evidence when they returned.

5.1.4 AI-Assisted Timetable Extraction

With an OpenAI-compatible key (a free Gemini key from Google AI Studio works through its compatible endpoint), an uploaded timetable photograph produced a proposed set of periods. The

administrator reviewed the proposal and applied it; nothing changed when the key was missing or the image was unreadable, so the feature is strictly optional.

5.1.5 Discussion

The results show that the evidence-based design meets the objectives: the dashboard never reports a seat as open without proof, the timetable boundaries are enforced without manual intervention, and the audit log makes every change traceable. The main limitations are the absence of HTTPS on the local network, the single administrator account and the fact that the lock overlay is enforced by a user-level agent rather than a system service; these are addressed in the conclusion as future work.

Table 5.1 summarises the verification performed against the functional requirements of Section 3.3.1. Each scenario was exercised by the automated suite and, where a user interaction was involved, repeated manually on the dashboard.

Table 5.1: Verification summary

Requirement	Verification performed	Result
Seat discovery and enrolment	Agent locates the server by UDP and enrols with a unique seat ID and token	Pass
Seat commands	Lock, unlock, message, sleep and reboot delivered within one poll	Pass
Grant expiry	Seat re-locks at expiry without administrator action	Pass
Timetable boundaries	Periods open and close on time with no roll-over of grants	Pass
Attendance allocation	Exactly N seats opened; shortage reported when seats are insufficient	Pass
Agent authentication	Wrong token rejected (401); duplicate seat refused (409)	Pass
Administrator authentication	No session rejected (401); login and failure audited	Pass
Wake-on-LAN	Magic packet sent; state changes only after a fresh heartbeat	Pass
Fail-closed behaviour	Server unreachable: seat remains locked and keeps retrying	Pass

5.2 Conclusion

LabNexus demonstrates that a college laboratory can be managed accurately with nothing beyond ordinary PCs, a staff laptop and the existing LAN. Routine work — preparation, locking,

timetable boundaries, attendance-driven allocation and Wake-on-LAN — all ran from a single dashboard, and every action was auditable at zero licence cost.

The system is a working prototype for a trusted LAN. Hardening it for permanent deployment would require HTTPS, token rotation, an auto-starting agent service, support for multiple administrators and a database in place of the JSON state file; these form the natural next stage of the work.

CHAPTER-6
APPENDIX, REFERENCES AND SOURCE CODE

6.1 Appendix

This appendix records the artefacts of the project and the procedure needed to install and operate the system.

6.1.1 Project Repository and Documentation

- **Source code and issue tracker:** https://github.com/abhigyant167/LabNexus
- **Node.js documentation:** https://nodejs.org/docs/
- **Express documentation:** https://expressjs.com/
- **Google Gemini API documentation:** https://ai.google.dev/gemini-api/docs

6.1.2 Installation Procedure

1. Install Node.js 18 or later on the administrator laptop and on every lab PC.
2. Copy the admin folder to the administrator laptop, open a command prompt in it and run npm install.
3. Start the server with npm start and open http://localhost:8000 in a browser.
4. Allow TCP port 8000 and UDP port 41888 through the firewall on the administrator laptop.
5. Copy the pc-agent folder to every lab PC and double-click START-AGENT.bat.
6. Sign in to the dashboard, confirm the weekly timetable and register the laboratory.

6.1.3 Operating Procedure

1. Open the dashboard and confirm that every seat is shown as LOCKED or UNKNOWN before the period begins.
2. At the start of a period, enter the number of students present and take attendance; LabNexus unlocks exactly that many seats.
3. Use Grant for a temporary extension, and Message to send a note to a seat.
4. End the period, or allow the timetable to end it; every seat re-locks and the audit log records the transition.

5. Use Sleep, Reboot or Wake as required; a seat is reported awake only after a fresh heartbeat.

6.2 References

[1] LabNexus project repository and source code, GitHub. [Online]. Available: https://github.com/abhigyant167/LabNexus

[2] OpenJS Foundation, “Node.js documentation.” [Online]. Available: https://nodejs.org/docs/

[3] OpenJS Foundation, “Express 4.x documentation.” [Online]. Available: https://expressjs.com/

[4] Google, “Gemini API documentation.” [Online]. Available: https://ai.google.dev/gemini-api/docs

[5] Jawaharlal Nehru Technological University Hyderabad, “B.Tech project report guidelines,” 2026.

6.3 Source Code

The following excerpts are taken from the working source tree and show the authentication, discovery and registration paths described in Chapter 4. The complete sources are supplied with the report and are available in the repository listed in Section 6.1.

```
// ------------------------------------------------------------
// admin/server.js (excerpt) - authenticated admin middleware
// ------------------------------------------------------------
function requireAdmin(req, res, next) {
  const cookie = req.headers.cookie || "";
  const token = cookie.split(";").map((c) => c.trim())
    .find((c) => c.startsWith("ln_session="))?.slice(11);
  if (!token || !state.sessionValid(token))
    return res.status(401).json({ error: "Not authorized. Log in first." });
  next();
}

// ------------------------------------------------------------
// admin/server.js (excerpt) - administrator login
// ------------------------------------------------------------
app.post("/api/auth/login", (req, res) => {
  const { password } = req.body || {};
  const token = state.login(password);
  if (!token) {
    state.audit("LOGIN_FAILED", { ip: req.ip });
    return res.status(401).json({ error: "Wrong password" });
  }
  state.audit("LOGIN_OK", { ip: req.ip });
  res.setHeader("Set-Cookie", `ln_session=${token}; HttpOnly; Path=/; SameSite=Strict; Max-Age=43200`);
  res.json({ ok: true });
});

// ------------------------------------------------------------
// admin/discovery.js (excerpt) - UDP auto-discovery responder
// ------------------------------------------------------------
const MAGIC = "LABNEXUS_DISCOVER_V1";
sock.on("message", (msg, rinfo) => {
  if (msg.toString() !== MAGIC) return;
  const reply = Buffer.from(JSON.stringify({
    labnexus: true, httpPort, ts: Date.now(),
  }));
  sock.send(reply, rinfo.port, rinfo.address, () => {});
});

// ------------------------------------------------------------
// pc-agent/agent.js (excerpt) - registration outcome handling
// ------------------------------------------------------------
const { status, body } = res;
if (status === 200 && body?.ok) {
  registerFailures = 0; registered = true;
  console.log(`Registered as PC-${PC_ID} at ${SERVER_URL}`);
  return "ok";
}
if (status === 409 || body?.code === "PC_ID_TAKEN") {
  // this config was copied from another laptop: enroll afresh
  await enroll(); return "transient";
}
if (status === 401) { await enroll(); return "transient"; }
```

CHAPTER-7
TEAM DETAILS

The project was carried out by the following team members of II B.Tech, Centre for Engineering Education Research, CMR College of Engineering & Technology.

Table 7.1: Team members and roll numbers

PHOTO	NAME	ROLL NUMBER
[PHOTO]	Abhigyan Tiwari	25H51A67D0
[PHOTO]	K. Bhargavi	25H51A67F7
[PHOTO]	N. Chandrika	25H51A67H2
[PHOTO]	B. Ashwitha	25H51A6716
[PHOTO]	K. Sri Vardhan	25H51A6717

Signature of the project guide: _____